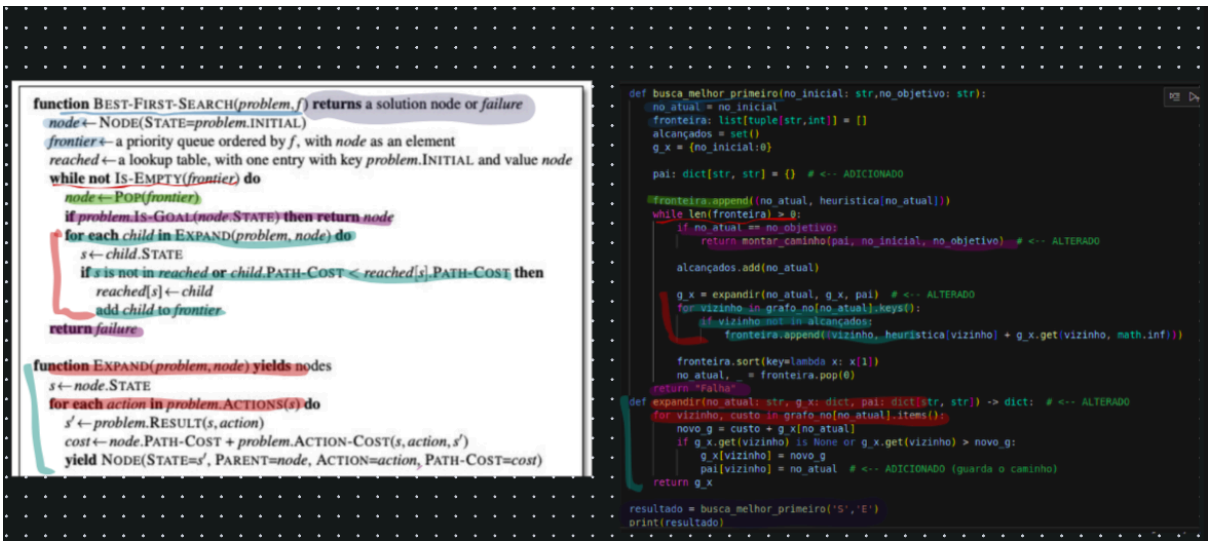
Foi usado o pseudo código base que o professor deu, para criar uma função a-estrela. linhas de correspondência por cores foram utilizadas para efeitos comparativos.



Depois disso foram usados engenharia de prompts para escalar o código a fim de se solucionar um problema.

Esse é o primeiro código com um grafo simples onde o a-estrela tem que apenas descobrir qual é o melhor caminho. ele foi feito “na mão” com apenas algumas poucas ajudas de ia. Já os o escalonamento para atividades mais robustas foram feitos com um uso maior de ia.

```
from typing import Any, Callable, Iterable, Optional, List, Tuple,
Dict, Set
import heapq                # fila de prioridade (UCS / A*)
from collections import deque  # fila (BFS)
import math                 # inf, etc.
import random
from time import sleep
from typing import Optional
```

```
import math
from typing import Optional

grafo_no = { 'S':{'A':3},
             'A':{'B':1, 'C':2,'D': 3},
             'B':{'E':4},
             'C':{'E':5},
             'D':{'E':1},
             'E':{}
}
```

```python
heuristica = {
    'S':5,
    'A':4,
    'B':3,
    'C':4,
    'D':1,
    'E':0
}

def montar_caminho(pai: dict[str, str], inicio: str, fim: str) ->
list[str]:
    caminho = [fim]
    atual = fim
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)
    caminho.reverse()
    return caminho

def busca_melhor_primeiro(no_inicial: str,no_objetivo: str):
    no_atual = no_inicial
    fronteira: list[tuple[str,int]] = []
    alcançados = set()
    g_x = {no_inicial:0}

    pai: dict[str, str] = {}  # <-- ADICIONADO

    fronteira.append((no_atual, heuristica[no_atual]))
    while len(fronteira) > 0:
        if no_atual == no_objetivo:
            return montar_caminho(pai, no_inicial, no_objetivo)  # <--
ALTERADO

        alcançados.add(no_atual)

        g_x = expandir(no_atual, g_x, pai)  # <-- ALTERADO
        for vizinho in grafo_no[no_atual].keys():
            if vizinho not in alcançados:
                fronteira.append((vizinho, heuristica[vizinho] +
g_x.get(vizinho, math.inf)))

        fronteira.sort(key=lambda x: x[1])
```

```
        no_atual, _ = fronteira.pop(0)

    return "Falha"


def expandir(no_atual: str, g_x: dict, pai: dict[str, str]) -> dict:  #
<-- ALTERADO
    for vizinho, custo in grafo_no[no_atual].items():
        novo_g = custo + g_x[no_atual]
        if g_x.get(vizinho) is None or g_x.get(vizinho) > novo_g:
            g_x[vizinho] = novo_g
            pai[vizinho] = no_atual  # <-- ADICIONADO (guarda o caminho)
    return g_x

resultado = busca_melhor_primeiro('S','E')
print(resultado)
```

prompt: preciso melhorar esse código. esse código já tem um grafo previamente definido. quero fazer um grafo seguindo a ideia de matriz quadrada. o programa inicia. pergunta qual a ordem da matriz do gráfico. se 3x3 ele cria os elementos do grafo seguindo a ideia de matriz 3x3

```
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
```

```python
# Geração automática do grafo N×N
# ------------------------------

def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False)
-> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade (matriz) n×n.
    Cada célula vira um nó.
    As arestas ligam o nó aos vizinhos (cima/baixo/esquerda/direita).

    Representação do grafo:
      grafo[no_atual] = {vizinho1: custo, vizinho2: custo, ...}

    Parâmetros:
      n         -> tamanho da grade (N para N×N)
      custo     -> custo fixo para andar de um nó ao vizinho
      diagonais -> se True, liga também as diagonais (8 vizinhos)

    Retorno:
      dicionário representando o grafo.
    """
    grafo: dict[str, dict[str, int]] = {}

    # Movimentos 4-vizinhos (cima, baixo, esquerda, direita)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # Se permitir diagonais, adiciona os 4 movimentos diagonais
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    # Percorre cada célula da matriz e cria suas ligações
    for i in range(n):
        for j in range(n):
            no = id_no(i, j)
            grafo[no] = {}

            # Para cada movimento possível, tenta criar a aresta para o
vizinho
            for di, dj in moves:
                ni, nj = i + di, j + dj

                # Só cria a conexão se o vizinho estiver dentro dos
limites da matriz
```

```
                if 0 <= ni < n and 0 <= nj < n:
                    grafo[no][id_no(ni, nj)] = custo

    return grafo

# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
```

```
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------

def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """

    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
```

```
        _, atual = heapq.heappop(fronteira)

        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
```

```
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')


# ----------------------------
# Programa principal
# ----------------------------

def main():
    # 1) Lê o tamanho da matriz N
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)

    # 2) Decide se o grafo terá conexões diagonais ou não
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    # 3) Cria o grafo automaticamente (grade N×N)
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais)

    # 4) Lê início e objetivo
    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio = ler_no("Nó inicial (linha,coluna): ", n)
    objetivo = ler_no("Nó objetivo (linha,coluna): ", n)

    # 5) Roda A* e imprime resultado
```

```python
    caminho, custo_total = a_estrela(grafo, inicio, objetivo)

    if caminho is None:
        print("\nFalha: não encontrou caminho.")
    else:
        print("\nCaminho encontrado:")
        print(" -> ".join(caminho))
        print("Custo total:", custo_total)



# Executa o main se este arquivo for rodado diretamente
if __name__ == "__main__":
    main()
```

prompt : eu consigo fazer ele mostrar o grafico em forma de matriz

```python
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------
```

```python
def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False)
-> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade (matriz) n×n.
    Cada célula vira um nó.
    As arestas ligam o nó aos vizinhos (cima/baixo/esquerda/direita).

    Representação do grafo:
      grafo[no_atual] = {vizinho1: custo, vizinho2: custo, ...}

    Parâmetros:
      n         -> tamanho da grade (N para N×N)
      custo     -> custo fixo para andar de um nó ao vizinho
      diagonais -> se True, liga também as diagonais (8 vizinhos)

    Retorno:
      dicionário representando o grafo.
    """
    grafo: dict[str, dict[str, int]] = {}

    # Movimentos 4-vizinhos (cima, baixo, esquerda, direita)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # Se permitir diagonais, adiciona os 4 movimentos diagonais
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    # Percorre cada célula da matriz e cria suas ligações
    for i in range(n):
        for j in range(n):
            no = id_no(i, j)
            grafo[no] = {}

            # Para cada movimento possível, tenta criar a aresta para o
vizinho
            for di, dj in moves:
                ni, nj = i + di, j + dj

                # Só cria a conexão se o vizinho estiver dentro dos
limites da matriz
                if 0 <= ni < n and 0 <= nj < n:
                    grafo[no][id_no(ni, nj)] = custo
```

```
    return grafo

# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)
```

```
    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------

def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """

    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
        _, atual = heapq.heappop(fronteira)
```

```
        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
```

```
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')

# ----------------------------
# Impressão da matriz e caminho encontrado
# ----------------------------
def imprimir_matriz_caminho(n: int, caminho: list[str] | None, inicio:
str, objetivo: str):
    """
    Imprime uma grade N×N no terminal:
      S = início
      G = objetivo
      * = caminho
      . = vazio
    """
    # Cria matriz com pontos
    grade = [["." for _ in range(n)] for _ in range(n)]

    # Marca caminho
    if caminho:
        for no in caminho:
            i, j = parse_no(no)
            grade[i][j] = "*"

    # Marca início e objetivo (por cima do caminho)
    si, sj = parse_no(inicio)
    oi, oj = parse_no(objetivo)
```

```
    grade[si][sj] = "S"
    grade[oi][oj] = "G"

    # Imprime com cabeçalho de colunas
    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()

# ----------------------------
# Programa principal
# ----------------------------

def main():
    # 1) Lê o tamanho da matriz N
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)

    # 2) Decide se o grafo terá conexões diagonais ou não
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    # 3) Cria o grafo automaticamente (grade N×N)
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais)

    # 4) Lê início e objetivo
    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio = ler_no("Nó inicial (linha,coluna): ", n)
    objetivo = ler_no("Nó objetivo (linha,coluna): ", n)

    # 5) Roda A* e imprime resultado
    caminho, custo_total = a_estrela(grafo, inicio, objetivo)

    if caminho is None:
        print("\nFalha: não encontrou caminho.")
    else:
        print("\nCaminho encontrado:")
        print(" -> ".join(caminho))
        print("Custo total:", custo_total)

    imprimir_matriz_caminho(n, caminho, inicio, objetivo)

# Executa o main se este arquivo for rodado diretamente
```

```
if __name__ == "__main__":
    main()
```

prompt: vamos seguir essa linha, e adicionar a ideia de pega pega, onde o destino se mexe aleatoriamente apenas um passo por vez , segue a ideia de dois jogadores.

```
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------

def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False)
-> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade (matriz) n×n.
    Cada célula vira um nó.
    As arestas ligam o nó aos vizinhos (cima/baixo/esquerda/direita).

    Representação do grafo:
      grafo[no_atual] = {vizinho1: custo, vizinho2: custo, ...}

    Parâmetros:
```

```
    n         -> tamanho da grade (N para N×N)
    custo     -> custo fixo para andar de um nó ao vizinho
    diagonais -> se True, liga também as diagonais (8 vizinhos)

  Retorno:
    dicionário representando o grafo.
  """
  grafo: dict[str, dict[str, int]] = {}

  # Movimentos 4-vizinhos (cima, baixo, esquerda, direita)
  moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

  # Se permitir diagonais, adiciona os 4 movimentos diagonais
  if diagonais:
      moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

  # Percorre cada célula da matriz e cria suas ligações
  for i in range(n):
      for j in range(n):
          no = id_no(i, j)
          grafo[no] = {}

          # Para cada movimento possível, tenta criar a aresta para o
vizinho
          for di, dj in moves:
              ni, nj = i + di, j + dj

              # Só cria a conexão se o vizinho estiver dentro dos
limites da matriz
              if 0 <= ni < n and 0 <= nj < n:
                  grafo[no][id_no(ni, nj)] = custo

  return grafo

# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
  """
  Heurística Manhattan (adequada para grade 4-vizinhos).
  Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.
```

```
    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------

def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
```

```
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """

    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
        _, atual = heapq.heappop(fronteira)

        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
```

```
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
```

```
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')

# ----------------------------
# Impressão da matriz e caminho encontrado
# ----------------------------
def imprimir_matriz_caminho(n: int, caminho: list[str] | None, inicio:
str, objetivo: str):
    """
    Imprime uma grade N×N no terminal:
      S = início
      G = objetivo
      * = caminho
      . = vazio
    """
    # Cria matriz com pontos
    grade = [["." for _ in range(n)] for _ in range(n)]

    # Marca caminho
    if caminho:
        for no in caminho:
            i, j = parse_no(no)
            grade[i][j] = "*"

    # Marca início e objetivo (por cima do caminho)
    si, sj = parse_no(inicio)
    oi, oj = parse_no(objetivo)
    grade[si][sj] = "S"
    grade[oi][oj] = "G"

    # Imprime com cabeçalho de colunas
    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()

# ----------------------------
# Simulação de Pega-Pega
```

```python
# ----------------------------


def vizinhos_validos(grafo: dict[str, dict[str, int]], no: str) ->
list[str]:
    """Retorna a lista de vizinhos válidos de um nó (com base no
grafo)."""
    return list(grafo[no].keys())

def mover_aleatorio(grafo: dict[str, dict[str, int]], pos: str, evitar:
str | None = None) -> str:
    """
    Move 1 passo aleatório para um vizinho.
    Se 'evitar' for dado, tenta não ir para esse nó (se houver
alternativa).
    """
    opcoes = vizinhos_validos(grafo, pos)
    if not opcoes:
        return pos

    if evitar is not None:
        opcoes_sem_evitar = [v for v in opcoes if v != evitar]
        if opcoes_sem_evitar:
            opcoes = opcoes_sem_evitar

    return random.choice(opcoes)

def imprimir_matriz_pega_pega(n: int, cacador: str, alvo: str, caminho:
list[str] | None = None):
    """
    Imprime a grade N×N:
      C = caçador
      A = alvo
      * = caminho planejado pelo caçador (opcional)
      . = vazio
    """
    grade = [["." for _ in range(n)] for _ in range(n)]

    if caminho:
        for no in caminho:
            i, j = parse_no(no)
            grade[i][j] = "*"
```

```
    ci, cj = parse_no(cacador)
    ai, aj = parse_no(alvo)

    # Marca alvo primeiro e depois o caçador (se estiverem no mesmo
lugar, mostra captura)
    grade[ai][aj] = "A"
    grade[ci][cj] = "C" if cacador != alvo else "X"  # X = capturou

    print("\n   " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()

def simular_pega_pega(
    n: int,
    diagonais: bool,
    inicio_cacador: str,
    inicio_alvo: str,
    max_turnos: int = 30,
    mostrar_caminho: bool = True,
    seed: int | None = None
):
    """
    Simula o pega-pega em uma grade N×N.

    Regras:
      - Caçador usa A* e anda 1 passo (o próximo nó do caminho).
      - Alvo se move 1 passo aleatoriamente (depois do caçador).
      - Captura se caçador == alvo em qualquer momento.

    Parâmetros:
      - max_turnos: limita a simulação pra não ficar infinita.
      - seed: fixa aleatoriedade (bom pra testes/demonstração).
    """
    if seed is not None:
        random.seed(seed)

    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais)

    cacador = inicio_cacador
    alvo = inicio_alvo
```

```
    print("\n=== INÍCIO DO PEGA-PEGA ===")
    imprimir_matriz_pega_pega(n, cacador, alvo)

    for turno in range(1, max_turnos + 1):
        # 1) Se já começou capturado
        if cacador == alvo:
            print(f"Captura imediata no turno {turno-1}!")
            return

        # 2) Caçador planeja (A*) até a posição ATUAL do alvo
        caminho, _ = a_estrela(grafo, cacador, alvo)

        print(f"Turno {turno}: C={cacador} | A={alvo}")
        imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None)

        if caminho is None or len(caminho) == 0:
            print(f"Turno {turno}: Caçador não conseguiu planejar
caminho até o alvo.")
            imprimir_matriz_pega_pega(n, cacador, alvo)
            return

        # 3) Caçador anda 1 passo (se caminho tem pelo menos 2 nós, o
próximo é caminho[1])
        if len(caminho) >= 2:
            proximo = caminho[1]
        else:
            proximo = cacador  # já está no alvo (caso raro)
        cacador = proximo

        # 4) Checa captura após movimento do caçador
        if cacador == alvo:
            print(f"Turno {turno}: Caçador capturou o alvo!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None)
            return

        # 5) Alvo move 1 passo aleatório
        # (opcionalmente tenta evitar ir pro nó do caçador)
        alvo = mover_aleatorio(grafo, alvo, evitar=cacador)

        # 6) Checa captura após movimento do alvo
        if cacador == alvo:
```

```
            print(f"Turno {turno}: Alvo caiu no caçador (captura)!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None)
            return



    print(f"Fim: não houve captura em {max_turnos} turnos.")
    imprimir_matriz_pega_pega(n, cacador, alvo)

def main():
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio_cacador = ler_no("Caçador (linha,coluna): ", n)
    inicio_alvo = ler_no("Alvo (linha,coluna): ", n)

    max_turnos = ler_int("Max de turnos (ex: 30): ", minimo=1)
    seed_txt = input("Seed (enter = aleatório / número = reproduzível):
").strip()
    seed = int(seed_txt) if seed_txt else None

    simular_pega_pega(
        n=n,
        diagonais=diagonais,
        inicio_cacador=inicio_cacador,
        inicio_alvo=inicio_alvo,
        max_turnos=max_turnos,
        mostrar_caminho=True,
        seed=seed
    )

if __name__ == "__main__":
    main()
```

prompt: agora vamos colocar a ideia de obstaculos na matriz para dificultar a captura do alvo. faça isso sem mexer na função a estrela.

```python
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------

def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False,
obstaculos: set[str] | None = None) -> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade n×n.
    Se 'obstaculos' for informado, esses nós NÃO existem no grafo (não
são visitáveis).
    """
    if obstaculos is None:
        obstaculos = set()

    grafo: dict[str, dict[str, int]] = {}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    for i in range(n):
        for j in range(n):
            no = id_no(i, j)
```

```
            # se o nó é obstáculo, ele não entra no grafo
            if no in obstaculos:
                continue

            grafo[no] = {}

            for di, dj in moves:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
                    viz = id_no(ni, nj)

                    # não cria aresta para obstáculo
                    if viz in obstaculos:
                        continue

                    grafo[no][viz] = custo

    return grafo


# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
```

```
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------

def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """
```

```
    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
        _, atual = heapq.heappop(fronteira)

        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
```

```python
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')




#-----------------------------
# Geração de obstáculos
#-----------------------------
```

```python
def gerar_obstaculos_aleatorios(n: int, qtd: int, inicio: str,
objetivo: str) -> set[str]:
    """
    Gera um conjunto de nós (strings "i,j") que serão obstáculos.
    Não coloca obstáculo no início nem no objetivo.
    """
    total = n * n
    maximo = total - 2  # tirando inicio e objetivo

    if qtd < 0:
        qtd = 0
    if qtd > maximo:
        qtd = maximo

    proibidos = {inicio, objetivo}

    # lista de todos os nós possíveis exceto os proibidos
    candidatos = [id_no(i, j) for i in range(n) for j in range(n) if
id_no(i, j) not in proibidos]

    # sorteia sem repetir
    return set(random.sample(candidatos, qtd))

# ----------------------------
# Simulação de Pega-Pega - Movimento aleatório do alvo (sem
planejamento, só para ter um comportamento "vivo") o caçador é o agente
inteligente que usa A* para perseguir o alvo, enquanto o alvo se move
aleatoriamente (com a opção de tentar evitar o caçador).
# ----------------------------


def vizinhos_validos(grafo: dict[str, dict[str, int]], no: str) ->
list[str]:
    """Retorna a lista de vizinhos válidos de um nó (com base no
grafo)."""
    return list(grafo[no].keys())

def mover_aleatorio(grafo: dict[str, dict[str, int]], pos: str, evitar:
str | None = None) -> str:
    """
    Move 1 passo aleatório para um vizinho.
    Se 'evitar' for dado, tenta não ir para esse nó (se houver
alternativa).
```

```
    """
    opcoes = vizinhos_validos(grafo, pos)
    if not opcoes:
        return pos

    if evitar is not None:
        opcoes_sem_evitar = [v for v in opcoes if v != evitar]
        if opcoes_sem_evitar:
            opcoes = opcoes_sem_evitar

    return random.choice(opcoes)

# ----------------------------
# Impressão da matriz e caminho encontrado. é colocado o caçador, o
alvo, os obstáculos e o caminho (se houver)
# ----------------------------
def imprimir_matriz_pega_pega(n: int, cacador: str, alvo: str, caminho:
list[str] | None = None, obstaculos: set[str] | None = None):
    """
    Imprime a grade N×N:
      C = caçador
      A = alvo
      * = caminho (opcional)
      # = obstáculo
      . = vazio
    """
    if obstaculos is None:
        obstaculos = set()

    grade = [["." for _ in range(n)] for _ in range(n)]

    # obstáculos
    for no in obstaculos:
        i, j = parse_no(no)
        grade[i][j] = "#"

    # caminho
    if caminho:
        for no in caminho:
            if no in obstaculos:
                continue
            i, j = parse_no(no)
            grade[i][j] = "*"
```

```
    ci, cj = parse_no(cacador)
    ai, aj = parse_no(alvo)

    grade[ai][aj] = "A"
    grade[ci][cj] = "C" if cacador != alvo else "X"

    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()

#-----------------------------
# Simulação de Pega-Pega
#-----------------------------
def simular_pega_pega(
    n: int,
    diagonais: bool,
    inicio_cacador: str,
    inicio_alvo: str,
    max_turnos: int = 30,
    mostrar_caminho: bool = True,
    seed: int | None = None,
    qtd_obstaculos: int = 0
):

    if seed is not None:
        random.seed(seed)

    # gera obstáculos aleatórios (não pega C nem A)
    obstaculos = gerar_obstaculos_aleatorios(n, qtd_obstaculos,
inicio_cacador, inicio_alvo)
    # cria o grafo já "bloqueando" esses nós
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)
    # verifica se início e alvo ainda são válidos (pode acontecer de um
obstáculo bloquear o início ou o alvo, então é bom avisar)
    if inicio_cacador not in grafo or inicio_alvo not in grafo:
        print("Erro: início ou alvo ficou inválido no grafo (obstáculos
bloquearam).")
        return
    # inicializa posições
```

```
    cacador = inicio_cacador
    alvo = inicio_alvo

    print("\n=== INÍCIO DO PEGA-PEGA ===")
    print("Legenda: C=Caçador | A=Alvo | #=Obstáculo | *=Caminho
planejado | .=Vazio")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)

    for turno in range(1, max_turnos + 1):
        # 1) Se já começou capturado
        if cacador == alvo:
            print(f"Captura imediata no turno {turno-1}!")
            return

        # 2) Caçador planeja (A*) até a posição ATUAL do alvo
        caminho, _ = a_estrela(grafo, cacador, alvo)

        print(f"Turno {turno}: C={cacador} | A={alvo}")
        imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)



        if caminho is None or len(caminho) == 0:
            print(f"Turno {turno}: Caçador não conseguiu planejar
caminho até o alvo.")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 3) Caçador anda 1 passo (se caminho tem pelo menos 2 nós, o
próximo é caminho[1])
        if len(caminho) >= 2:
            proximo = caminho[1]
        else:
            proximo = cacador  # já está no alvo (caso raro)
        cacador = proximo

        # 4) Checa captura após movimento do caçador
        if cacador == alvo:
            print(f"Turno {turno}: Caçador capturou o alvo!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
```

```
            return

        # 5) Alvo move 1 passo aleatório
        # (opcionalmente tenta evitar ir pro nó do caçador)
        alvo = mover_aleatorio(grafo, alvo, evitar=cacador)

        # 6) Checa captura após movimento do alvo
        if cacador == alvo:
            print(f"Turno {turno}: Alvo caiu no caçador (captura)!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

            return

    print(f"Fim: não houve captura em {max_turnos} turnos.")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)


#-----------------------------
# Função principal para rodar a simulação
#-----------------------------

def main():
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio_cacador = ler_no("Caçador (linha,coluna): ", n)
    inicio_alvo = ler_no("Alvo (linha,coluna): ", n)

    max_turnos = ler_int("Max de turnos (ex: 30): ", minimo=1)
    seed_txt = input("Seed (enter = aleatório / número = reproduzível):
").strip()
    seed = int(seed_txt) if seed_txt else None
    qtd_obstaculos = ler_int("Quantos obstáculos aleatórios você quer?
", minimo=0)

    simular_pega_pega(
        n=n,
        diagonais=diagonais,
        inicio_cacador=inicio_cacador,
        inicio_alvo=inicio_alvo,
```

```
        max_turnos=max_turnos,
        mostrar_caminho=True,
        seed=seed,
        qtd_obstaculos=qtd_obstaculos
    )

if __name__ == "__main__":
    main()
```

prompt: quero que o número de obstáculos aumente uma unidade a cada turno.

```
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------

def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False,
obstaculos: set[str] | None = None) -> dict[str, dict[str, int]]:
    """
```

```
    Cria um grafo em grade n×n.
    Se 'obstaculos' for informado, esses nós NÃO existem no grafo (não
são visitáveis).
    """
    if obstaculos is None:
        obstaculos = set()

    grafo: dict[str, dict[str, int]] = {}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    for i in range(n):
        for j in range(n):
            no = id_no(i, j)

            # se o nó é obstáculo, ele não entra no grafo
            if no in obstaculos:
                continue

            grafo[no] = {}

            for di, dj in moves:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
                    viz = id_no(ni, nj)

                    # não cria aresta para obstáculo
                    if viz in obstaculos:
                        continue

                    grafo[no][viz] = custo

    return grafo


# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
```

```
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------
```

```python
def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """

    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
        _, atual = heapq.heappop(fronteira)

        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]
```

```
        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
```

```python
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')




#------------------------------
# Geração de obstáculos
#------------------------------

def gerar_obstaculos_aleatorios(n: int, qtd: int, inicio: str,
objetivo: str) -> set[str]:
    """
    Gera um conjunto de nós (strings "i,j") que serão obstáculos.
    Não coloca obstáculo no início nem no objetivo.
    """
    total = n * n
    maximo = total - 2  # tirando inicio e objetivo

    if qtd < 0:
        qtd = 0
    if qtd > maximo:
        qtd = maximo

    proibidos = {inicio, objetivo}

    # lista de todos os nós possíveis exceto os proibidos
    candidatos = [id_no(i, j) for i in range(n) for j in range(n) if
id_no(i, j) not in proibidos]

    # sorteia sem repetir
    return set(random.sample(candidatos, qtd))

# ------------------------------
# Simulação de Pega-Pega - Movimento aleatório do alvo (sem
planejamento, só para ter um comportamento "vivo") o caçador é o agente
```

```
inteligente que usa A* para perseguir o alvo, enquanto o alvo se move
aleatoriamente (com a opção de tentar evitar o caçador).
# ----------------------------


def vizinhos_validos(grafo: dict[str, dict[str, int]], no: str) ->
list[str]:
    """Retorna a lista de vizinhos válidos de um nó (com base no
grafo)."""
    return list(grafo[no].keys())

def mover_aleatorio(grafo: dict[str, dict[str, int]], pos: str, evitar:
str | None = None) -> str:
    """
    Move 1 passo aleatório para um vizinho.
    Se 'evitar' for dado, tenta não ir para esse nó (se houver
alternativa).
    """
    opcoes = vizinhos_validos(grafo, pos)
    if not opcoes:
        return pos

    if evitar is not None:
        opcoes_sem_evitar = [v for v in opcoes if v != evitar]
        if opcoes_sem_evitar:
            opcoes = opcoes_sem_evitar

    return random.choice(opcoes)

# ----------------------------
# Impressão da matriz e caminho encontrado. é colocado o caçador, o
alvo, os obstáculos e o caminho (se houver)
# ----------------------------
def imprimir_matriz_pega_pega(n: int, cacador: str, alvo: str, caminho:
list[str] | None = None, obstaculos: set[str] | None = None):
    """
    Imprime a grade N×N:
      C = caçador
      A = alvo
      * = caminho (opcional)
      # = obstáculo
      . = vazio
    """
```

```
    if obstaculos is None:
        obstaculos = set()

    grade = [["." for _ in range(n)] for _ in range(n)]

    # obstáculos
    for no in obstaculos:
        i, j = parse_no(no)
        grade[i][j] = "#"

    # caminho
    if caminho:
        for no in caminho:
            if no in obstaculos:
                continue
            i, j = parse_no(no)
            grade[i][j] = "*"

    ci, cj = parse_no(cacador)
    ai, aj = parse_no(alvo)

    grade[ai][aj] = "A"
    grade[ci][cj] = "C" if cacador != alvo else "X"

    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()

#-----------------------------
# Simulação de Pega-Pega
#-----------------------------
def simular_pega_pega(
    n: int,
    diagonais: bool,
    inicio_cacador: str,
    inicio_alvo: str,
    max_turnos: int = 30,
    mostrar_caminho: bool = True,
    seed: int | None = None,
    qtd_obstaculos: int = 0
):
```

```
    if seed is not None:
        random.seed(seed)

    # quantidade fixa de obstáculos
    qtd_obs = qtd_obstaculos if qtd_obstaculos >= 0 else 0

     # sorteia obstáculos iniciais (não pega C nem A)
    obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs, inicio_cacador,
inicio_alvo)

    # limita a quantidade de obstáculos para não bloquear tudo (pode ser
ajustado)
    max_obs = n * n - 2  # não pode ocupar início e alvo

    # cria o grafo já "bloqueando" esses nós
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)

    # verifica se início e alvo ainda são válidos (pode acontecer de um
obstáculo bloquear o início ou o alvo, então é bom avisar)
    if inicio_cacador not in grafo or inicio_alvo not in grafo:
        print("Erro: início ou alvo ficou inválido no grafo (obstáculos
bloquearam).")
        return

    # inicializa posições
    cacador = inicio_cacador
    alvo = inicio_alvo

    print("\n=== INÍCIO DO PEGA-PEGA ===")
    print("Legenda: C=Caçador | A=Alvo | #=Obstáculo | *=Caminho
planejado | .=Vazio")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)

    for turno in range(1, max_turnos + 1):

        # aumenta a quantidade de obstáculos a cada turno
        qtd_obs = min(qtd_obs + 1, max_obs)
        # 0) Obstáculos mudam de lugar aleatoriamente (mantém a mesma
quantidade)
        obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs, cacador,
alvo)
```

```
        grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)

        # 1) Se já começou capturado
        if cacador == alvo:
            print(f"Captura imediata no turno {turno-1}!")
            return

        # 2) Caçador planeja (A*) até a posição ATUAL do alvo
        caminho, _ = a_estrela(grafo, cacador, alvo)

        print(f"Turno {turno} | Obstáculos: {qtd_obs} | C={cacador} |
A={alvo}")
        imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

        if caminho is None or len(caminho) == 0:
            print(f"Turno {turno}: Caçador não conseguiu planejar
caminho até o alvo.")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 3) Caçador anda 1 passo (se caminho tem pelo menos 2 nós, o
próximo é caminho[1])
        if len(caminho) >= 2:
            proximo = caminho[1]
        else:
            proximo = cacador  # já está no alvo (caso raro)
        cacador = proximo

        # 4) Checa captura após movimento do caçador
        if cacador == alvo:
            print(f"Turno {turno}: Caçador capturou o alvo!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 5) Alvo move 1 passo aleatório
        # (opcionalmente tenta evitar ir pro nó do caçador)
        alvo = mover_aleatorio(grafo, alvo, evitar=cacador)

        # 6) Checa captura após movimento do alvo
```

```
        if cacador == alvo:
            print(f"Turno {turno}: Alvo caiu no caçador (captura)!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

            return

    print(f"Fim: não houve captura em {max_turnos} turnos.")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)


#-----------------------------
# Função principal para rodar a simulação
#-----------------------------

def main():
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio_cacador = ler_no("Caçador (linha,coluna): ", n)
    inicio_alvo = ler_no("Alvo (linha,coluna): ", n)

    max_turnos = ler_int("Max de turnos (ex: 30): ", minimo=1)
    seed_txt = input("Seed (enter = aleatório / número = reproduzível):
").strip()
    seed = int(seed_txt) if seed_txt else None
    qtd_obstaculos = ler_int("Quantos obstáculos aleatórios você quer?
", minimo=0)

    simular_pega_pega(
        n=n,
        diagonais=diagonais,
        inicio_cacador=inicio_cacador,
        inicio_alvo=inicio_alvo,
        max_turnos=max_turnos,
        mostrar_caminho=True,
        seed=seed,
        qtd_obstaculos=qtd_obstaculos
    )

if __name__ == "__main__":
```

```
    main()
```

prompt: quero que seja opcional aumentar o número de obstáculos e tbm se os obstáculos mudam de lugar ou não

```
# ----------------------------
# Utilitários de nó (ID e parse)
# ----------------------------

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"

def parse_no(s: str) -> tuple[int, int]:
    """
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------

def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False,
obstaculos: set[str] | None = None) -> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade n×n.
    Se 'obstaculos' for informado, esses nós NÃO existem no grafo (não
são visitáveis).
    """
    if obstaculos is None:
        obstaculos = set()

    grafo: dict[str, dict[str, int]] = {}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    for i in range(n):
        for j in range(n):
            no = id_no(i, j)

            # se o nó é obstáculo, ele não entra no grafo
            if no in obstaculos:
                continue

            grafo[no] = {}

            for di, dj in moves:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
                    viz = id_no(ni, nj)

                    # não cria aresta para obstáculo
                    if viz in obstaculos:
                        continue

                    grafo[no][viz] = custo

    return grafo


# ----------------------------
# Heurística (para A*)
# ----------------------------

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)
```

```python
# ------------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ------------------------------

def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ------------------------------
# Algoritmo A* (A-estrela)
# ------------------------------

def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
```

```
    - fechados: nós já processados (para não reprocessar)

  Retorno:
    (caminho, custo_total) se encontrar
    (None, None) se não encontrar
  """

  # Heap de prioridades: cada item é (f, nó)
  fronteira = []
  heapq.heappush(fronteira, (0, inicio))

  # Guarda o melhor pai (de onde veio) para cada nó
  pai: dict[str, str] = {}

  # g[n] = custo do melhor caminho do início até n
  g: dict[str, int] = {inicio: 0}

  # Conjunto de nós já "fechados" (expandidos)
  fechados: set[str] = set()

  while fronteira:
      # Pega o nó com menor f(n)
      _, atual = heapq.heappop(fronteira)

      # Se já foi fechado, ignora (evita trabalho duplicado)
      if atual in fechados:
          continue
      fechados.add(atual)

      # Se chegamos no objetivo, reconstrói e retorna
      if atual == objetivo:
          return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

      # Explora vizinhos do nó atual
      for vizinho, custo_aresta in grafo[atual].items():
          # custo para chegar no vizinho passando pelo atual
          novo_g = g[atual] + custo_aresta

          # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
          if novo_g < g.get(vizinho, math.inf):
              g[vizinho] = novo_g
```

```
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))

    # Se esvaziou a fronteira, não existe caminho
    return None, None

# ----------------------------
# Entrada robusta (lendo N e nós)
# ----------------------------

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")

def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')
```

```python
#-----------------------------
# Geração de obstáculos
#-----------------------------

def gerar_obstaculos_aleatorios(n: int, qtd: int, inicio: str,
objetivo: str) -> set[str]:
    """
    Gera um conjunto de nós (strings "i,j") que serão obstáculos.
    Não coloca obstáculo no início nem no objetivo.
    """
    total = n * n
    maximo = total - 2  # tirando inicio e objetivo

    if qtd < 0:
        qtd = 0
    if qtd > maximo:
        qtd = maximo

    proibidos = {inicio, objetivo}

    # lista de todos os nós possíveis exceto os proibidos
    candidatos = [id_no(i, j) for i in range(n) for j in range(n) if
id_no(i, j) not in proibidos]

    # sorteia sem repetir
    return set(random.sample(candidatos, qtd))

# -----------------------------
# Simulação de Pega-Pega - Movimento aleatório do alvo (sem
planejamento, só para ter um comportamento "vivo") o caçador é o agente
inteligente que usa A* para perseguir o alvo, enquanto o alvo se move
aleatoriamente (com a opção de tentar evitar o caçador).
# -----------------------------


def vizinhos_validos(grafo: dict[str, dict[str, int]], no: str) ->
list[str]:
    """Retorna a lista de vizinhos válidos de um nó (com base no
grafo)."""
    return list(grafo[no].keys())
```

```python
def mover_aleatorio(grafo: dict[str, dict[str, int]], pos: str, evitar:
str | None = None) -> str:
    """
    Move 1 passo aleatório para um vizinho.
    Se 'evitar' for dado, tenta não ir para esse nó (se houver
alternativa).
    """
    opcoes = vizinhos_validos(grafo, pos)
    if not opcoes:
        return pos

    if evitar is not None:
        opcoes_sem_evitar = [v for v in opcoes if v != evitar]
        if opcoes_sem_evitar:
            opcoes = opcoes_sem_evitar

    return random.choice(opcoes)

# ----------------------------
# Impressão da matriz e caminho encontrado. é colocado o caçador, o
alvo, os obstáculos e o caminho (se houver)
# ----------------------------
def imprimir_matriz_pega_pega(n: int, cacador: str, alvo: str, caminho:
list[str] | None = None, obstaculos: set[str] | None = None):
    """
    Imprime a grade N×N:
      C = caçador
      A = alvo
      * = caminho (opcional)
      # = obstáculo
      . = vazio
    """
    if obstaculos is None:
        obstaculos = set()

    grade = [["." for _ in range(n)] for _ in range(n)]

    # obstáculos
    for no in obstaculos:
        i, j = parse_no(no)
        grade[i][j] = "#"
```

```
    # caminho
    if caminho:
        for no in caminho:
            if no in obstaculos:
                continue
            i, j = parse_no(no)
            grade[i][j] = "*"

    ci, cj = parse_no(cacador)
    ai, aj = parse_no(alvo)

    grade[ai][aj] = "A"
    grade[ci][cj] = "C" if cacador != alvo else "X"

    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
    print()


#-----------------------------
# Função para adicionar obstáculos gradualmente (crescentes)
#-----------------------------

def adicionar_obstaculos_ate(
    n: int,
    obstaculos: set[str],
    qtd_desejada: int,
    proibidos: set[str]
) -> set[str]:
    """
    Mantém os obstáculos existentes e só adiciona novos até atingir
qtd_desejada.
    Não coloca em 'proibidos' (ex: caçador e alvo).
    """
    # candidatos = células livres e permitidas
    livres = [
        id_no(i, j)
        for i in range(n)
        for j in range(n)
        if id_no(i, j) not in obstaculos and id_no(i, j) not in
proibidos
```

```
    ]

    faltam = qtd_desejada - len(obstaculos)
    if faltam <= 0:
        return obstaculos

    # se não tem espaço, adiciona o máximo possível
    k = min(faltam, len(livres))
    novos = set(random.sample(livres, k))
    return obstaculos | novos




#-----------------------------
# Simulação de Pega-Pega
#-----------------------------
def simular_pega_pega(
    n: int,
    diagonais: bool,
    inicio_cacador: str,
    inicio_alvo: str,
    max_turnos: int = 30,
    mostrar_caminho: bool = True,
    seed: int | None = None,
    qtd_obstaculos: int = 0,
    obstaculos_moveis: bool = False,
    obstaculos_crescentes: bool = False
):

    if seed is not None:
        random.seed(seed)

    # quantidade fixa de obstáculos
    qtd_obs = qtd_obstaculos if qtd_obstaculos >= 0 else 0

    # limita a quantidade de obstáculos para não bloquear tudo (pode ser
ajustado)
    max_obs = n * n - 2  # não pode ocupar início e alvo

    # sorteia obstáculos iniciais (não pega C nem A)
    obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs, inicio_cacador,
inicio_alvo)
```

```
    # cria o grafo já "bloqueando" esses nós
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)

    # verifica se início e alvo ainda são válidos (pode acontecer de um
obstáculo bloquear o início ou o alvo, então é bom avisar)
    if inicio_cacador not in grafo or inicio_alvo not in grafo:
        print("Erro: início ou alvo ficou inválido no grafo (obstáculos
bloquearam).")
        return

    # inicializa posições
    cacador = inicio_cacador
    alvo = inicio_alvo

    print("\n=== INÍCIO DO PEGA-PEGA ===")
    print("Legenda: C=Caçador | A=Alvo | #=Obstáculo | *=Caminho
planejado | .=Vazio")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)

    for turno in range(1, max_turnos + 1):

        # 0) Atualiza obstáculos conforme opções do usuário
        # 0) Atualiza obstáculos conforme opções

        # 0.1) Se forem móveis: re-sorteia tudo (movem mesmo)
        if obstaculos_moveis:
             # se forem crescentes, aumenta antes de re-sorteiar
             if obstaculos_crescentes:
                 qtd_obs = min(qtd_obs + 1, max_obs)

             obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs,
cacador, alvo)

        # 0.2) Se NÃO forem móveis, mas forem crescentes: só adiciona
(não move os antigos)
        elif obstaculos_crescentes:
             qtd_obs = min(qtd_obs + 1, max_obs)
             obstaculos =
adicionar_obstaculos_ate(n=n,obstaculos=obstaculos,qtd_desejada=qtd_obs
,proibidos={cacador, alvo})
```

```
        # Sempre que obstáculo muda (móvel ou crescendo), refaz o grafo
        if obstaculos_moveis or obstaculos_crescentes:
             grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)



        # 1) Se já começou capturado
        if cacador == alvo:
            print(f"Captura imediata no turno {turno-1}!")
            return

        # 2) Caçador planeja (A*) até a posição ATUAL do alvo
        caminho, _ = a_estrela(grafo, cacador, alvo)

        mov_txt = "sim" if obstaculos_moveis else "não"
        cres_txt = "sim" if obstaculos_crescentes else "não"
        print(f"Turno {turno} | Obstáculos: {qtd_obs} | Móveis:
{mov_txt} | Crescentes: {cres_txt} | C={cacador} | A={alvo}")

        imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

        if caminho is None or len(caminho) == 0:
            print(f"Turno {turno}: Caçador não conseguiu planejar
caminho até o alvo.")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 3) Caçador anda 1 passo (se caminho tem pelo menos 2 nós, o
próximo é caminho[1])
        if len(caminho) >= 2:
            proximo = caminho[1]
        else:
            proximo = cacador  # já está no alvo (caso raro)
        cacador = proximo

        # 4) Checa captura após movimento do caçador
        if cacador == alvo:
            print(f"Turno {turno}: Caçador capturou o alvo!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
```

```
            return

        # 5) Alvo move 1 passo aleatório
        # (opcionalmente tenta evitar ir pro nó do caçador)
        alvo = mover_aleatorio(grafo, alvo, evitar=cacador)

        # 6) Checa captura após movimento do alvo
        if cacador == alvo:
            print(f"Turno {turno}: Alvo caiu no caçador (captura)!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

            return

    print(f"Fim: não houve captura em {max_turnos} turnos.")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)


#-----------------------------
# Função principal para rodar a simulação
#-----------------------------

def main():
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio_cacador = ler_no("Caçador (linha,coluna): ", n)
    inicio_alvo = ler_no("Alvo (linha,coluna): ", n)

    max_turnos = ler_int("Max de turnos (ex: 30): ", minimo=1)
    seed_txt = input("Seed (enter = aleatório / número = reproduzível):
").strip()
    seed = int(seed_txt) if seed_txt else None
    qtd_obstaculos = ler_int("Quantos obstáculos aleatórios você quer?
", minimo=0)
    obst_moveis = input("Obstáculos mudam de posição a cada turno?
(s/n): ").strip().lower() == "s"
    obst_crescentes = input("Quantidade de obstáculos aumenta a cada
turno? (s/n): ").strip().lower() == "s"

    simular_pega_pega(
```

```
        n=n,
        diagonais=diagonais,
        inicio_cacador=inicio_cacador,
        inicio_alvo=inicio_alvo,
        max_turnos=max_turnos,
        mostrar_caminho=True,
        seed=seed,
        qtd_obstaculos=qtd_obstaculos,
        obstaculos_moveis=obst_moveis,
        obstaculos_crescentes=obst_crescentes
    )

if __name__ == "__main__":
    main()
```

prompt: apenas uma organização do código

```
from __future__ import annotations

from typing import Any, Callable, Iterable, Optional, List, Tuple,
Dict, Set
from collections import deque  # fila (BFS)
import heapq                   # fila de prioridade (UCS / A*)
import math                    # inf, etc.
import random
from time import sleep
# ============================================================
# Entrada robusta (lendo N e nós)
# ============================================================

def id_no(linha: int, coluna: int) -> str:
    """
    Converte (linha, coluna) em um identificador único de nó.
    Ex: (0, 2) -> "0,2"
    """
    return f"{linha},{coluna}"


def parse_no(s: str) -> tuple[int, int]:
    """
```

```
    Faz o caminho inverso: "linha,coluna" -> (linha, coluna).
    Ex: "3,1" -> (3, 1)
    """
    a, b = s.split(",")
    return int(a), int(b)

# ----------------------------
# Geração automática do grafo N×N
# ----------------------------


# ============================================================
# Estrutura do problema (grade, vizinhança)
# ============================================================


def criar_grafo_grade(n: int, custo: int = 1, diagonais: bool = False,
obstaculos: set[str] | None = None) -> dict[str, dict[str, int]]:
    """
    Cria um grafo em grade n×n.
    Se 'obstaculos' for informado, esses nós NÃO existem no grafo (não
são visitáveis).
    """
    if obstaculos is None:
        obstaculos = set()

    grafo: dict[str, dict[str, int]] = {}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    if diagonais:
        moves += [(-1, -1), (-1, 1), (1, -1), (1, 1)]

    for i in range(n):
        for j in range(n):
            no = id_no(i, j)

            # se o nó é obstáculo, ele não entra no grafo
            if no in obstaculos:
                continue

            grafo[no] = {}

            for di, dj in moves:
```

```
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < n:
                    viz = id_no(ni, nj)

                    # não cria aresta para obstáculo
                    if viz in obstaculos:
                        continue

                    grafo[no][viz] = custo

    return grafo

def vizinhos_validos(grafo: dict[str, dict[str, int]], no: str) ->
list[str]:
    """Retorna a lista de vizinhos válidos de um nó (com base no
grafo)."""
    return list(grafo[no].keys())


# ============================================================
# Heurística (para A*)
# ============================================================

def heuristica_manhattan(no: str, objetivo: str) -> int:
    """
    Heurística Manhattan (adequada para grade 4-vizinhos).
    Mede "quantos passos mínimos" faltam (sem obstáculos) para chegar no
objetivo.

    h(n) = |x1-x2| + |y1-y2|
    """
    x1, y1 = parse_no(no)
    x2, y2 = parse_no(objetivo)
    return abs(x1 - x2) + abs(y1 - y2)

# ----------------------------
# Reconstrução do caminho (a partir do dicionário pai)
# ----------------------------


def reconstruir_caminho(pai: dict[str, str], inicio: str, objetivo:
str) -> list[str]:
    """
```

```
    Reconstrói o caminho do objetivo até o início usando o dicionário
'pai'.
    'pai[v]' guarda de qual nó viemos para chegar em 'v'.

    Ex:
      pai["0,1"] = "0,0"
      pai["0,2"] = "0,1"
      ...
    """
    caminho = [objetivo]
    atual = objetivo

    # Vai voltando pelos pais até chegar no início
    while atual != inicio:
        atual = pai[atual]
        caminho.append(atual)

    # Como construímos do fim pro começo, invertendo temos o caminho
correto
    caminho.reverse()
    return caminho

# ----------------------------
# Algoritmo A* (A-estrela)
# ----------------------------


def a_estrela(grafo: dict[str, dict[str, int]], inicio: str, objetivo:
str) -> tuple[list[str], int] | tuple[None, None]:
    """
    Executa o A* para encontrar o menor caminho entre 'inicio' e
'objetivo'.

    Estruturas principais:
      - fronteira (heap): guarda nós a explorar, priorizando menor f(n)
      - g[n]: custo real do início até n
      - pai[n]: predecessor usado para reconstruir o caminho
      - fechados: nós já processados (para não reprocessar)

    Retorno:
      (caminho, custo_total) se encontrar
      (None, None) se não encontrar
    """
```

```
    # Heap de prioridades: cada item é (f, nó)
    fronteira = []
    heapq.heappush(fronteira, (0, inicio))

    # Guarda o melhor pai (de onde veio) para cada nó
    pai: dict[str, str] = {}

    # g[n] = custo do melhor caminho do início até n
    g: dict[str, int] = {inicio: 0}

    # Conjunto de nós já "fechados" (expandidos)
    fechados: set[str] = set()

    while fronteira:
        # Pega o nó com menor f(n)
        _, atual = heapq.heappop(fronteira)

        # Se já foi fechado, ignora (evita trabalho duplicado)
        if atual in fechados:
            continue
        fechados.add(atual)

        # Se chegamos no objetivo, reconstrói e retorna
        if atual == objetivo:
            return reconstruir_caminho(pai, inicio, objetivo),
g[objetivo]

        # Explora vizinhos do nó atual
        for vizinho, custo_aresta in grafo[atual].items():
            # custo para chegar no vizinho passando pelo atual
            novo_g = g[atual] + custo_aresta

            # Relaxamento: se esse caminho é melhor do que o conhecido,
atualiza
            if novo_g < g.get(vizinho, math.inf):
                g[vizinho] = novo_g
                pai[vizinho] = atual

                # f(n) = g(n) + h(n)
                f = novo_g + heuristica_manhattan(vizinho, objetivo)
                heapq.heappush(fronteira, (f, vizinho))
```

```
    # Se esvaziou a fronteira, não existe caminho
    return None, None




# ============================================================
# Gerar (obstáculos, movimentos)
# ============================================================

def gerar_obstaculos_aleatorios(n: int, qtd: int, inicio: str,
objetivo: str) -> set[str]:
    """
    Gera um conjunto de nós (strings "i,j") que serão obstáculos.
    Não coloca obstáculo no início nem no objetivo.
    """
    total = n * n
    maximo = total - 2  # tirando inicio e objetivo

    if qtd < 0:
        qtd = 0
    if qtd > maximo:
        qtd = maximo

    proibidos = {inicio, objetivo}

    # lista de todos os nós possíveis exceto os proibidos
    candidatos = [id_no(i, j) for i in range(n) for j in range(n) if
id_no(i, j) not in proibidos]

    # sorteia sem repetir
    return set(random.sample(candidatos, qtd))


# Movimento aleatório do alvo (sem planejamento, só para ter um
comportamento "vivo") o caçador é o agente inteligente que usa A* para
perseguir o alvo, enquanto o alvo se move aleatoriamente (com a opção
de tentar evitar o caçador).

def mover_aleatorio(grafo: dict[str, dict[str, int]], pos: str, evitar:
str | None = None) -> str:
    """
    Move 1 passo aleatório para um vizinho.
```

```
    Se 'evitar' for dado, tenta não ir para esse nó (se houver
alternativa).
    """
    opcoes = vizinhos_validos(grafo, pos)
    if not opcoes:
        return pos

    if evitar is not None:
        opcoes_sem_evitar = [v for v in opcoes if v != evitar]
        if opcoes_sem_evitar:
            opcoes = opcoes_sem_evitar

    return random.choice(opcoes)



#============================================================
# Função para adicionar obstáculos gradualmente (crescentes)
#============================================================

def adicionar_obstaculos_ate(
    n: int,
    obstaculos: set[str],
    qtd_desejada: int,
    proibidos: set[str]
) -> set[str]:
    """
    Mantém os obstáculos existentes e só adiciona novos até atingir
qtd_desejada.
    Não coloca em 'proibidos' (ex: caçador e alvo).
    """
    # candidatos = células livres e permitidas
    livres = [
        id_no(i, j)
        for i in range(n)
        for j in range(n)
        if id_no(i, j) not in obstaculos and id_no(i, j) not in
proibidos
    ]

    faltam = qtd_desejada - len(obstaculos)
    if faltam <= 0:
        return obstaculos
```

```python
    # se não tem espaço, adiciona o máximo possível
    k = min(faltam, len(livres))
    novos = set(random.sample(livres, k))
    return obstaculos | novos


# ============================================================
# Entrada/Saída (CLI e impressão)
# ============================================================

def ler_int(msg: str, minimo: int = 1) -> int:
    """
    Lê um inteiro do usuário, garantindo que seja >= minimo.
    Repete até o usuário digitar corretamente.
    """
    while True:
        try:
            v = int(input(msg).strip())
            if v >= minimo:
                return v
            print(f"Digite um inteiro >= {minimo}.")
        except ValueError:
            print("Entrada inválida. Digite um número inteiro.")


def ler_no(msg: str, n: int) -> str:
    """
    Lê um nó no formato 'linha,coluna' e valida se está dentro de
0..n-1.
    Repete até o usuário digitar corretamente.
    """
    while True:
        s = input(msg).strip()
        try:
            i, j = parse_no(s)
            if 0 <= i < n and 0 <= j < n:
                return id_no(i, j)
            print(f"Fora do limite. Use 0..{n-1} em linha e coluna.")
        except Exception:
            print('Formato inválido. Use "linha,coluna" (ex: 0,0).')
```

```python
# ----------------------------
# Impressão da matriz e caminho encontrado. é colocado o caçador, o
alvo, os obstáculos e o caminho (se houver)
# ----------------------------

def imprimir_matriz_pega_pega(n: int, cacador: str, alvo: str, caminho:
list[str] | None = None, obstaculos: set[str] | None = None):
    """
    Imprime a grade N×N:
      C = caçador
      A = alvo
      * = caminho (opcional)
      # = obstáculo
      . = vazio
    """
    if obstaculos is None:
        obstaculos = set()

    grade = [["." for _ in range(n)] for _ in range(n)]

    # obstáculos
    for no in obstaculos:
        i, j = parse_no(no)
        grade[i][j] = "#"

    # caminho
    if caminho:
        for no in caminho:
            if no in obstaculos:
                continue
            i, j = parse_no(no)
            grade[i][j] = "*"

    ci, cj = parse_no(cacador)
    ai, aj = parse_no(alvo)

    grade[ai][aj] = "A"
    grade[ci][cj] = "C" if cacador != alvo else "X"

    print("\n    " + " ".join(f"{c:2d}" for c in range(n)))
    for i in range(n):
        print(f"{i:2d} " + " ".join(f"{grade[i][j]:2s}" for j in
range(n)))
```

```
    print()




# ============================================================
# Execução (simulação e main)
# ============================================================

#-----------------------------
# Simulação de Pega-Pega
#-----------------------------
def simular_pega_pega(
    n: int,
    diagonais: bool,
    inicio_cacador: str,
    inicio_alvo: str,
    max_turnos: int = 30,
    mostrar_caminho: bool = True,
    seed: int | None = None,
    qtd_obstaculos: int = 0,
    obstaculos_moveis: bool = False,
    obstaculos_crescentes: bool = False
):

    if seed is not None:
        random.seed(seed)

    # quantidade fixa de obstáculos
    qtd_obs = qtd_obstaculos if qtd_obstaculos >= 0 else 0

    # limita a quantidade de obstáculos para não bloquear tudo (pode ser
ajustado)
    max_obs = n * n - 2  # não pode ocupar início e alvo

    # sorteia obstáculos iniciais (não pega C nem A)
    obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs, inicio_cacador,
inicio_alvo)

    # cria o grafo já "bloqueando" esses nós
    grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)
```

```
    # verifica se início e alvo ainda são válidos (pode acontecer de um
obstáculo bloquear o início ou o alvo, então é bom avisar)
    if inicio_cacador not in grafo or inicio_alvo not in grafo:
        print("Erro: início ou alvo ficou inválido no grafo (obstáculos
bloquearam).")
        return

    # inicializa posições
    cacador = inicio_cacador
    alvo = inicio_alvo

    print("\n=== INÍCIO DO PEGA-PEGA ===")
    print("Legenda: C=Caçador | A=Alvo | #=Obstáculo | *=Caminho
planejado | .=Vazio")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)

    for turno in range(1, max_turnos + 1):

        # 0) Atualiza obstáculos conforme opções do usuário
        # 0) Atualiza obstáculos conforme opções

        # 0.1) Se forem móveis: re-sorteia tudo (movem mesmo)
        if obstaculos_moveis:
            # se forem crescentes, aumenta antes de re-sorteiar
            if obstaculos_crescentes:
                qtd_obs = min(qtd_obs + 1, max_obs)

            obstaculos = gerar_obstaculos_aleatorios(n, qtd_obs,
cacador, alvo)

        # 0.2) Se NÃO forem móveis, mas forem crescentes: só adiciona
(não move os antigos)
        elif obstaculos_crescentes:
            qtd_obs = min(qtd_obs + 1, max_obs)
            obstaculos =
adicionar_obstaculos_ate(n=n,obstaculos=obstaculos,qtd_desejada=qtd_obs
,proibidos={cacador, alvo})

        # Sempre que obstáculo muda (móvel ou crescendo), refaz o grafo
        if obstaculos_moveis or obstaculos_crescentes:
            grafo = criar_grafo_grade(n, custo=1, diagonais=diagonais,
obstaculos=obstaculos)
```

```
        # 1) Se já começou capturado
        if cacador == alvo:
            print(f"Captura imediata no turno {turno-1}!")
            return

        # 2) Caçador planeja (A*) até a posição ATUAL do alvo
        caminho, _ = a_estrela(grafo, cacador, alvo)

        mov_txt = "sim" if obstaculos_moveis else "não"
        cres_txt = "sim" if obstaculos_crescentes else "não"
        print(f"Turno {turno} | Obstáculos: {qtd_obs} | Móveis:
{mov_txt} | Crescentes: {cres_txt} | C={cacador} | A={alvo}")

        imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

        if caminho is None or len(caminho) == 0:
            print(f"Turno {turno}: Caçador não conseguiu planejar
caminho até o alvo.")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 3) Caçador anda 1 passo (se caminho tem pelo menos 2 nós, o
próximo é caminho[1])
        if len(caminho) >= 2:
            proximo = caminho[1]
        else:
            proximo = cacador  # já está no alvo (caso raro)
        cacador = proximo

        # 4) Checa captura após movimento do caçador
        if cacador == alvo:
            print(f"Turno {turno}: Caçador capturou o alvo!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)
            return

        # 5) Alvo move 1 passo aleatório
        # (opcionalmente tenta evitar ir pro nó do caçador)
        alvo = mover_aleatorio(grafo, alvo, evitar=cacador)
```

```
        # 6) Checa captura após movimento do alvo
        if cacador == alvo:
            print(f"Turno {turno}: Alvo caiu no caçador (captura)!")
            imprimir_matriz_pega_pega(n, cacador, alvo, caminho if
mostrar_caminho else None, obstaculos)

            return

    print(f"Fim: não houve captura em {max_turnos} turnos.")
    imprimir_matriz_pega_pega(n, cacador, alvo, None, obstaculos)


#-----------------------------
# Função principal para rodar a simulação
#-----------------------------


def main():
    n = ler_int("Qual a ordem da matriz (N para N×N)? ", minimo=2)
    diagonais = input("Permitir diagonais? (s/n): ").strip().lower() ==
"s"

    print("\nNós válidos vão de 0..N-1. Ex: 0,0  |  2,1  |  N-1,N-1\n")
    inicio_cacador = ler_no("Caçador (linha,coluna): ", n)
    inicio_alvo = ler_no("Alvo (linha,coluna): ", n)

    max_turnos = ler_int("Max de turnos (ex: 30): ", minimo=1)
    seed_txt = input("Seed (enter = aleatório / número = reproduzível):
").strip()
    seed = int(seed_txt) if seed_txt else None
    qtd_obstaculos = ler_int("Quantos obstáculos aleatórios você quer?
", minimo=0)
    obst_moveis = input("Obstáculos mudam de posição a cada turno?
(s/n): ").strip().lower() == "s"
    obst_crescentes = input("Quantidade de obstáculos aumenta a cada
turno? (s/n): ").strip().lower() == "s"

    simular_pega_pega(
        n=n,
        diagonais=diagonais,
        inicio_cacador=inicio_cacador,
        inicio_alvo=inicio_alvo,
```

```
        max_turnos=max_turnos,
        mostrar_caminho=True,
        seed=seed,
        qtd_obstaculos=qtd_obstaculos,
        obstaculos_moveis=obst_moveis,
        obstaculos_crescentes=obst_crescentes
    )

if __name__ == "__main__":
    main()
```